

Smart Campus Management System

CS-104L Object-Oriented Programming

HITEC University Taxila

Date: 05-06-2026

GitHub URL: <https://github.com/ahmadoffcl/OOP>

Group Members

- Ahmad Ali | Reg No: 25-CS-067
- Umer Altaf | Reg No: 25-CS-057
- Muhammed Ahmad | Reg No: 25-CS-252

Introduction

This project is a simple command-line Smart Campus Management System written in C++. It manages basic campus records for students, faculty, courses, library items, fee records, hostel rooms, and reports. The purpose is to show all major OOP concepts from the CS-104L course in one connected program. The program uses simple arrays, pointers, classes, file handling, and exception handling. The design is intentionally easy to read so each group member can explain the code during viva.

System Modules

- Person module: Person, Student, GradStudent, Faculty, and Staff classes.
- PersonManager: saved person records with add, view, delete, save, and reload options.
- Course module: Course, Enrollment, and CourseManager classes with saved course/enrollment records and capacity checking.
- Library module: Book and Journal catalog with add/search/delete, issue/return, overdue fine, file handling, and arrays.
- Finance module: FeeRecord, Invoice, and FinanceManager classes with saved fee records, payments, fines, invoices, copy handling, and static invoice counter.
- Hostel module: Room, HostelBlock, and HostelManager with saved rooms, saved allocations, composition, and multiple inheritance.
- Reports module: full campus summary, GPA sorting, roll-number search, overdue records, utility info, campus text report, and PDF-style text report generation.

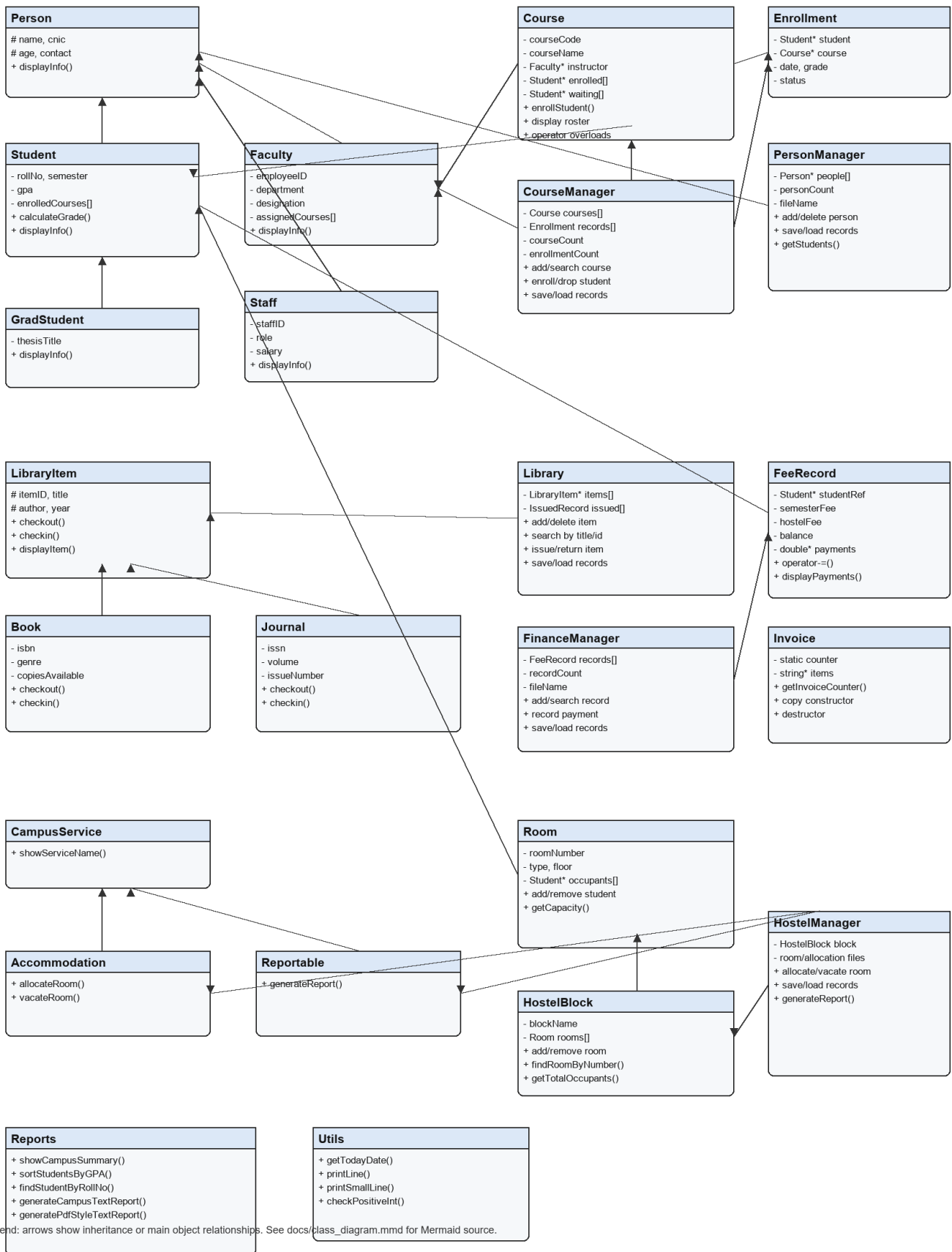
Class Diagram

The class diagram below shows the main classes and relationships used in the project. Inheritance is used in Person, LibraryItem, and hostel interfaces. Composition is used where HostelManager contains HostelBlock, and aggregation is used where Course stores a Faculty pointer without owning the faculty object.

Smart Campus Management System (SCMS)

Smart Campus Management System - Class Diagram

HITEC University Taxila | CS-104L OOP



OOP Concepts Implementation

1. Classes and Objects: All modules define classes and make objects in main.cpp.

```
Student s1(...); Course oop(...);
```

2. Encapsulation: Private data is changed through public getters, setters, and member functions.

```
string getRollNo() const; void setGPA(double newGPA);
```

3. Default Constructor: Several classes provide simple empty/default object creation.

```
Student::Student() : Person(), semester(1), gpa(0.0) {}
```

4. Parameterized Constructor: Objects can be created with real starting data.

```
Course::Course(string code, string name, int credits, Faculty* teacher, int capacity)
```

5. Copy Constructor: Person, Course, FeeRecord, and Invoice copy object data.

```
FeeRecord::FeeRecord(const FeeRecord& other)
```

6. Destructors: Library, Invoice, and HostelManager have destructors for cleanup.

```
Invoice::~~Invoice() { delete[] items; }
```

7. Single Inheritance: Student reuses common Person data and behavior.

```
class Student : public Person
```

8. Multi-level Inheritance: GradStudent extends Student, which already extends Person.

```
class GradStudent : public Student
```

9. Multiple Inheritance: HostelManager implements accommodation and report behavior.

```
class HostelManager : public Accommodation, public Reportable
```

10. Virtual Inheritance: Accommodation and Reportable virtually inherit CampusService to avoid duplicate base parts.

```
class Accommodation : virtual public CampusService
```

11. Abstract Classes: Person, LibraryItem, Accommodation, and Reportable cannot be directly instantiated.

```
virtual void displayInfo() const = 0;
```

12. Runtime Polymorphism: Base pointers call the correct derived displayInfo function.

```
Person* people[MAX_PEOPLE]; people[i]->displayInfo();
```

13. Operator Overloading ==: Course objects are compared by course code.

```
bool Course::operator==(const Course& other) const
```

14. Operator Overloading <<: Course and Invoice can be printed using stream insertion.

```
ostream& operator<<(ostream& out, const Course& course)
```

15. Operator Overloading +: Two course waiting lists are merged.

```
Student** Course::operator+(Course& other)
```

16. Operator Overloading -=: FeeRecord records a payment and reduces balance.

```
fee -= 25000;
```

17. Friend Functions: operator<< is declared as a friend for Course and Invoice.

```
friend ostream& operator<<(ostream& out, const Invoice& invoice);
```

18. Static Members: Invoice has one shared invoiceCounter for all invoice objects.

```
int Invoice::invoiceCounter = 0;
```

19. Deep Copy: FeeRecord and Invoice copy dynamic arrays instead of sharing them.

```
payments = new double[paymentCount];
```

Smart Campus Management System (SCMS)

20. Search Functions: Library and hostel use loop searches, and Reports uses `std::find_if` for roll number search.

```
Student* Reports::findStudentByRollNo(Student* students[], int count, string rollNo)
```

21. Array-based Collections: The project uses arrays for people, courses, library items, rooms, and students.

```
Person* people[MAX_PEOPLE];
```

22. Arrays of Objects: HostelBlock keeps Room objects in an array.

```
Room rooms[MAX_BLOCK_ROOMS];
```

23. Exception Handling: Custom exceptions are thrown and caught for capacity and overdue cases.

```
throw CapacityExceededException("Course is full");
```

24. File I/O: Person, course, enrollment, library, finance, hostel, and report data are loaded/saved using `fstream`.

```
ofstream file(fileName);
```

25. Reporting and Utilities: Reports and Utils keep campus summaries, report files, date, formatting, and validation helpers separate.

```
Reports::showCampusSummary(...);
```

26. Memory Management: Objects and arrays created with `new` are deleted using `delete` or `delete[]`.

```
delete[] sortedStudents;
```

27. Sorting and Searching: Reports sorts students by GPA, searches by roll number, and lists overdue library records.

```
sortStudentsByGPA(students, count);
```

28. Composition: HostelManager owns a HostelBlock object.

```
HostelBlock block;
```

29. Aggregation: Course stores Faculty* and Room stores Student* without owning those people.

```
Faculty* instructor; Student* occupants[];
```

Smart Campus Management System (SCMS)

Module Descriptions

- Module 1 - Person Hierarchy: Stores common personal information in Person and uses Student, GradStudent, Faculty, and Staff subclasses. PersonManager stores saved records, supports input, deletion, save, and reload.
- Module 2 - Course and Enrollment: Stores saved course data, enrolls saved students, tracks waiting-list records, saves/reloads text files, blocks over-capacity enrollment, and demonstrates overloaded operators.
- Module 3 - Library System: Stores books and journals, adds/deletes catalog items, searches by title or ID, saves/loads catalog and issued records, tracks issued items, and handles overdue fines.
- Module 4 - Fee and Finance: Stores saved fee records, records payments with operator=, adds fines, deep-copies fee data, and generates invoices with a static counter.
- Module 5 - Hostel Management: Uses saved rooms, saved allocations, room search/add/delete, multiple inheritance, virtual inheritance, and composition to allocate/vacate rooms.
- Module 6 - Reporting and Utilities: Shows full campus summary, sorts/searches student data, lists overdue library records, writes consolidated text/PDF-style reports, and keeps helper functions separate.

GitHub Workflow

The project is already a local git repository with separate commits for setup, documentation, and each major module. The public GitHub URL is included in README.md and on the title page of this report. The GitHub Actions workflow compiles the code and runs a smoke test through the console menus.

- Local branch: main
- Current local commits: see git log --oneline in the repository.
- A GitHub Actions build workflow is included at .github/workflows/build.yml for the bonus compile check.
- Do not submit until the public GitHub link is shared through the course portal.

Testing Summary

- Build command .\build.bat completed successfully.
- All six home menu modules were tested.
- Screen-based navigation was tested with Enter pauses after action pages.
- Module 1 CRUD was tested: add, display, save, reload, delete, save, and reload.
- Module 1 add-course action was tested by adding CS-200 to 25-CS-067.
- Module 2 added a course, searched it, enrolled students, sent a full-course student to waiting list, showed roster, dropped a student, saved/reloaded records, compared courses, and merged waiting lists.
- Library module added book/journal records, searched by title and ID, issued and returned items, blocked duplicate issue, showed overdue fine, deleted an item, and saved/reloaded records.
- Finance module added/search fee records, recorded payment, added fine, generated invoice, showed copy constructor/assignment, and saved/reloaded records.
- Hostel module loaded saved rooms, added/searched/deleted a room, allocated/vacated a saved student, blocked duplicate allocation, saved/reloaded records, and printed occupancy report.
- Reports module showed full campus summary, sorted/searched students, listed overdue library records, displayed utility helpers, and created data/campus_report.txt.
- Reports module created data/campus_pdf_report.txt with full system summary.
- Wrong input test with abc did not freeze the program.

Smart Campus Management System (SCMS)

- Full captured output files are saved in docs/test_outputs.

Module1 Person Output Screenshot

```
Module 1 - Person Hierarchy

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
PERSON MODULE
-----
1. Show all saved people
2. Show saved students
3. Add Student
4. Add Grad Student
5. Add Faculty
6. Add Staff
7. Add course to student
8. Delete person by ID
9. Save records
10. Reload records
0. Back to Home
Enter choice:
-----
SAVED PERSON RECORDS
-----

Student Name: Ahmad Ali
Roll No: 25-CS-067
Semester: 2
GPA: 3.4
Grade: B
Courses: No course added

... see module1_person.txt for full captured output ...
```

Module1 Crud Test Output Screenshot

```
Module 1 - Person CRUD

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
PERSON MODULE
-----
1. Show all saved people
2. Show saved students
3. Add Student
4. Add Grad Student
5. Add Faculty
6. Add Staff
7. Add course to student
8. Delete person by ID
9. Save records
10. Reload records
0. Back to Home
Enter choice:
-----
ADD STUDENT
-----
Name: CNIC: Age: Contact: Roll No: Semester: GPA: Student added. Use option 9 to save records.

Press Enter to continue...
-----
PERSON MODULE
-----
1. Show all saved people

... see module1_crud_test.txt for full captured output ...
```

Module1 Add Course Test Output Screenshot

```
Module 1 - Add Course to Student

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
PERSON MODULE
-----
1. Show all saved people
2. Show saved students
3. Add Student
4. Add Grad Student
5. Add Faculty
6. Add Staff
7. Add course to student
8. Delete person by ID
9. Save records
10. Reload records
0. Back to Home
Enter choice:
-----
ADD COURSE TO STUDENT
-----
Student Roll No: Course Code: Course added to student for this session.

Press Enter to continue...
-----
PERSON MODULE
-----
1. Show all saved people

... see module1_add_course_test.txt for full captured output ...
```

Module2 Course Output Screenshot

```
Module 2 - Course and Enrollment

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:

-----
COURSE AND ENROLLMENT MODULE
-----
1. Show all courses
2. Add course
3. Search course by code
4. Enroll student in course
5. Show course roster
6. Drop student from course
7. Compare two courses using ==
8. Merge waiting lists using +
9. Show enrollment records
10. Save course/enrollment records
11. Reload course/enrollment records
0. Back to Home
Enter choice:

-----
ADD COURSE
-----
Saved Faculty:
1. F-101 | Sir Usman | Computer Science
-----
Course Code: Course Name: Credit Hours: Max Capacity: Instructor Employee ID: Course added. Use option 10 to save r

Press Enter to continue...

... see module2_course.txt for full captured output ...
```

Module3 Library Output Screenshot

```
Module 3 - Library System

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
LIBRARY MODULE
-----
1. Show catalog
2. Add Book
3. Add Journal
4. Search by title
5. Search by item ID
6. Issue item
7. Return item
8. Show issued records
9. Delete item by ID
10. Save library records
11. Reload library records
0. Back to Home
Enter choice:
-----
LIBRARY CATALOG
-----

--- Library Catalog ---

Book ID: B001
Title: C++ Basics
Author: Deitel

... see module3_library.txt for full captured output ...
```

Module4 Finance Output Screenshot

```
Module 4 - Fee and Finance

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
FEE AND FINANCE MODULE
-----
1. Show all fee records
2. Add fee record
3. Search fee record by roll no
4. Record payment using -=
5. Add library fine
6. Show payment history
7. Generate invoice
8. Show FeeRecord copy constructor/assignment
9. Save finance records
10. Reload finance records
0. Back to Home
Enter choice:
-----
SAVED FEE RECORDS
-----

--- Fee Record ---
Student: Ahmad Ali
Roll No: 25-CS-067
Semester Fee: 65000
Hostel Fee: 20000
Library Fine: 500

... see module4_finance.txt for full captured output ...
```

Module5 Hostel Output Screenshot

```
Module 5 - Hostel Management

-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:
-----
HOSTEL MODULE
-----
1. Show hostel service name
2. Show all rooms
3. Add room
4. Search room by number
5. Allocate student to room
6. Auto allocate student
7. Vacate student room
8. Show student's room
9. Show hostel summary
10. Show occupancy report
11. Save hostel records
12. Reload hostel records
13. Delete empty room
0. Back to Home
Enter choice:
-----
HOSTEL SERVICE
-----
Hostel Accommodation Service

Press Enter to continue...
-----

... see module5_hostel.txt for full captured output ...
```

Module6 Reports Output Screenshot

```
Module 6 - Reports and Utilities

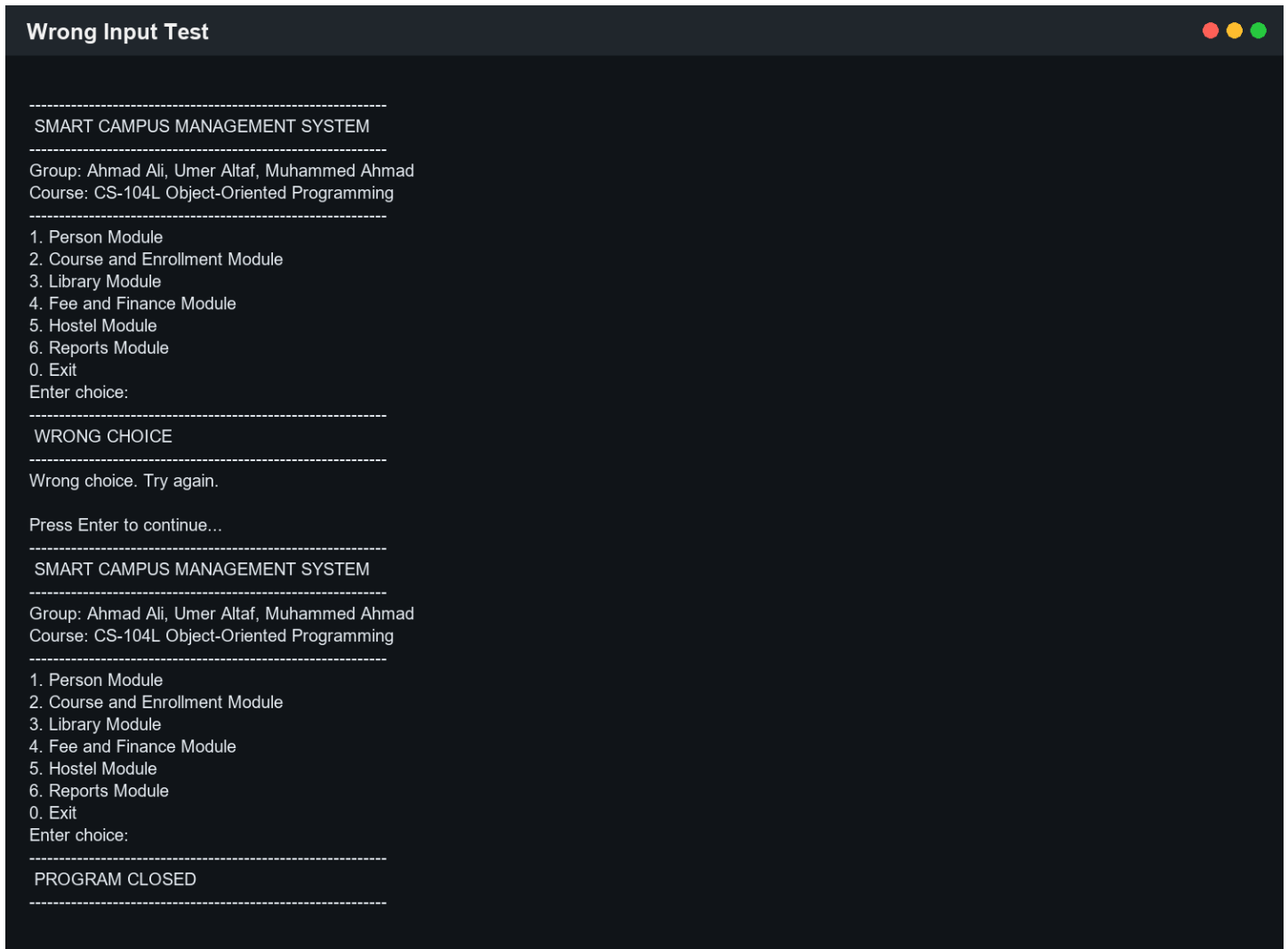
-----
SMART CAMPUS MANAGEMENT SYSTEM
-----
Group: Ahmad Ali, Umer Altaf, Muhammed Ahmad
Course: CS-104L Object-Oriented Programming
-----
1. Person Module
2. Course and Enrollment Module
3. Library Module
4. Fee and Finance Module
5. Hostel Module
6. Reports Module
0. Exit
Enter choice:

-----
REPORTS MODULE
-----
1. Show complete campus summary
2. Sort and show students by GPA
3. Find student by roll number
4. Show top GPA student
5. Show overdue library records
6. Generate consolidated text report
7. Generate consolidated PDF-style report
8. Show utility/date helper info
0. Back to Home
Enter choice:

-----
COMPLETE CAMPUS SUMMARY
-----
Date: 05-06-2026
-----
Total People: 6
Total Students: 4
Saved Courses: 1
Enrollment Records: 2
Library Items: 2
Issued Records: 1
Currently Issued Items: 0

... see module6_reports.txt for full captured output ...
```

Wrong Input Output Screenshot



Smart Campus Management System (SCMS)

Challenges and Solutions

- Understanding polymorphism: solved by using Person* array in main.cpp.
- Managing dynamic memory: solved by deleting Library items and Invoice item arrays.
- Handling full course capacity: solved by using a custom exception class.
- File handling format: solved by using simple pipe-separated text data.

Conclusion

This project helped practice core C++ OOP concepts in a practical campus system. The project is intentionally console-based and simple so each module can be explained in viva. Before final submission, the group should review the code, test the menus once more, and make sure each member can explain the module they worked on.

References

- Course project brief provided by HITEC University Taxila.
- C++ class notes and lab practice.
- cppreference.com for basic C++ syntax reference.